manages associations. The associated object needs an ID so that the parent object can link to it, and in Rails you get an ActiveRecord ID only when an ActiveRecord instance is saved to the database, meaning that a true association can exist only if the subordinate object is in the database and has an ID.

The easiest way to manage subordinate objects is by using build_stubbed to create the parent. Since the build_stubbed strategy assigns an ID to the objects being created, using build_stubbed sidesteps the whole issue. If a factory with associations is instantiated using build_stubbed, then by default all the associations are also invoked using build_stubbed. That solves the problem as long as you always use build_stubbed.

Calling the parent factory (in this case Task) with create causes the associated factory (in this case Project) to also be instantiated using create.

Where it gets tricky is if you use the build strategy. By default, even if you call the parent factory with build, the subordinate factory is still called with create. This is necessary to make sure the subordinate object has a Rails ID.

As a result, even if you use the build strategy specifically to avoid slow and unnecessary database interaction, if the factory has associations you'll still save objects to the database. Since those associated factories may themselves have associations, if you aren't careful you can end up saving a lot of objects to the database, resulting in prohibitively slow tests.

It's exactly this characteristic of factory_bot that has made it unwelcome in some circles, particularly if the people in those circles have to maintain large, unwieldy test suites. Factory-association misuse can be a big cause of a slow test suite, as tests create many more objects than they need to because of factory_bot associations.

However, if you use the longer form of specifying the association, you can explicitly specify the build strategy:

```
FactoryBot.define do
  factory :task do
    title: "To Something"
    size: 3
    project
    association :doer, factory: :user, strategy: :build
  end
end
```

If you go this route, you may have problems because the associated object won't have an ID. In this specific case, for example, the Task object will have its user attribute set but not its user_id. If your code is expecting the user_id to